

# Trabalho Prático 03 – MLPClassifier

GCC128–InteligênciaArtificial | Prof.Ahmed Ali Abdalla Esmin  
Aluno: Tobias Maugus Bueno Cougo

## 1. Metodologia

Aplicou-se o **MLPClassifier** do Scikit-learn às bases **Iris** e **Wine**, em um Pipeline com **StandardScaler** (ajustado só no treino, evitando vazamento de dados) seguido de uma rede com 1 camada oculta de 16 neurônios, ativação ReLU e solver L-BFGS (adequado para bases pequenas). Cada base foi dividida em 70% treino / 30% teste, estratificada, com **random\_state=42** — a mesma divisão e semente usadas no TP01 (KNN) no Iris, permitindo comparação direta entre os dois algoritmos.

## 2. Resultados

**Tabela 1.** Métricas do MLPClassifier nas duas bases.

| Base | Treino / Teste | Acurácia | Precisão macro | Revocação macro |
|------|----------------|----------|----------------|-----------------|
| Iris | 105 / 45       | 0,9111   | 0,9298         | 0,9111          |
| Wine | 124 / 54       | 0,9630   | 0,9619         | 0,9619          |

No Iris, o único erro sistemático foi virginica confundida com versicolor (4 das 15 amostras). No Wine, houve apenas 1 troca em cada sentido entre class\_1 e class\_2 (20/21 e 14/15 corretas); class\_0 foi 100% correta em ambas as bases/classes equivalentes.

**Tabela 2.** Comparação direta no Iris: KNN (TP01, k=5) vs. MLPClassifier — mesma divisão e semente.

| Métrica         | KNN TP01 (k=5) | MLPClassifier | Diferença (MLP – KNN) |
|-----------------|----------------|---------------|-----------------------|
| Acurácia        | 0,9778         | 0,9111        | -0,0667               |
| Precisão macro  | 0,9792         | 0,9298        | -0,0494               |
| Revocação macro | 0,9778         | 0,9111        | -0,0667               |

## 3. Análise e Conclusão

**Iris — KNN levou vantagem:** nesta configuração específica, o KNN (k=5) superou o MLP em todas as métricas (Tabela 2). A base Iris é pequena (150 amostras) e geometricamente bem separada, condição em que um método baseado em distância, sem hiperparâmetros para ajustar, já captura bem a estrutura dos dados. O MLP usado tem arquitetura fixa (16 neurônios), sem busca de hiperparâmetros — com um conjunto de teste de apenas 45 amostras, poucos erros a mais (aqui, 4) já derrubam vários pontos percentuais, então essa diferença não permite concluir superioridade geral de um algoritmo.

**Wine — MLP com bom desempenho:** com mais atributos e escalas distintas entre eles, o MLPClassifier atingiu 96,30% de acurácia, evidenciando que a padronização dentro do Pipeline foi eficaz e que a rede consegue lidar bem com um problema multiclasse de maior dimensionalidade — sem um baseline de KNN no TP01 para essa base, a comparação direta não é possível.

**Trade-offs entre os algoritmos:** o KNN é um método "preguiçoso" — praticamente sem fase de treino, mas com custo de predição proporcional ao tamanho da base, já que compara cada nova amostra com todos os exemplos armazenados. O MLP inverte esse custo: aprende pesos durante o treino (mais caro) e prediz rapidamente depois, via operações matriciais — mas exige padronização dos atributos e escolha de hiperparâmetros (arquitetura, ativação, solver).

**Conclusão:** o experimento não aponta um algoritmo universalmente superior. A escolha entre KNN e MLP deve considerar o tamanho da base, o custo de treino/predição aceitável, a necessidade de padronização e ajuste de hiperparâmetros, e a estabilidade dos resultados em novas divisões dos dados — não apenas a métrica de um único experimento.

**Link do repositório no GitHub:** <https://github.com/TobiasMaugus/IA-Projects>

**Link do vídeo de apresentação:** <https://youtu.be/jxDJdKzt8gg>